Introduzione 2

Argomenti

- **Oggetti**
  - tipologie di oggetti
  - creare/nominare un oggetto
  - tipologie di dato
- **Operatori**
  - matematici
  - relazionali
  - logici

Oggetti

Oggetti

Tutto quello che possiamo creare in R viene definito oggetto (e.g., numeri, vettori, matrici, funzioni).

```r
numero = 4
numero
```

```
[1] 4
```

```r
vettore = c(1,2,3,4)
vettore
```

```
[1] 1 2 3 4
```

```r
matrice = matrix(nrow = 2, ncol = 2, data = vettore)
matrice
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
```

Funzioni

Tutto quello che facciamo in R è chiamare **funzioni** su oggetti.

Le funzioni ci permettono di **creare** e **modificare** oggetti.

```
vettore
```

```
[1] 1 2 3 4
```

```
mean(x = vettore)
```

```
[1] 2.5
```

```
numero
```

```
[1] 4
```

```
numero = mean(vettore)
numero
```

```
[1] 2.5
```

Creare/nominare oggetti

Gli oggetti si possono creare e tramite il comando **<-** oppure **=**

```
x1 = 3 # nome = oggetto
x1
```

```
[1] 3
```

```
x2 <- 3 # nome <- oggetto
x2
```

```
[1] 3
```

```
x1 == x2 # i due oggetti sono identici?
```

```
[1] TRUE
```

Voi potete scegliere il comando che preferite, l'importante è essere consistenti.

Regole sulla denominazione di oggetti

- Deve iniziare con una lettera e può contenere lettere, numeri, underscore (_), o punti (.).
- Potrebbe anche iniziare con un punto (.) ma in tal caso non può essere seguito da un numero.

```
.3 = 3
```

```
Error in `0.3 = 3`:
! invalid (do_set) left-hand side to assignment
```

```
.x = 3
```

- Non deve contenere caratteri speciali come #, &, $, ?, etc.
- Non deve essere una parola riservata ovvero quelle parole che sono utilizzate da R con un significato speciale (?reserved).

```r
function = 3
```

```
Error in parse(text = input): <text>:1:10: unexpected '='
1: function =
             ^
```

```r
TRUE = 3
```

```
Error in `TRUE = 3`:
! invalid (do_set) left-hand side to assignment
```

```r
if = 3
```

```
Error in parse(text = input): <text>:1:4: unexpected '='
1: if =
       ^
```

Convenzioni

Ci sono alcuni nomi che non sono proibiti ma sono sconsigliati

```
T
```

```
[1] TRUE
```

```
TRUE
```

```
[1] TRUE
```

```
T == TRUE
```

```
[1] TRUE
```

```
T = 1; T
```

```
[1] 1
```

```
T == TRUE
```

```
[1] TRUE
```

```
sum(2,3)
```

```
[1] 5
```

```
sum = 4; sum
```

```
[1] 4
```

L'uso di "*.*" nei nomi degli oggetti (ad esempio, "my.data") va bene in **R** ma non è consentito in **Python**, dove "*.*" fa parte della sintassi del linguaggio.

Tra i diversi linguaggi, le *convenzioni di denominazione* per i nomi di variabili più lunghi e composti da più parole privilegiano `snake_case` (ad esempio, "**my_data**") o `camelCase` (ad esempio, "**myData**"), e **abbreviazioni** dove appropriato (ad esempio, "**unipdData**" meglio di "university_of_padova_dataset").

R è case-sensitive

```r
Nome = "Margherita"

nome = "margherita"

Nome
```

```
[1] "Margherita"
```

```r
nome
```

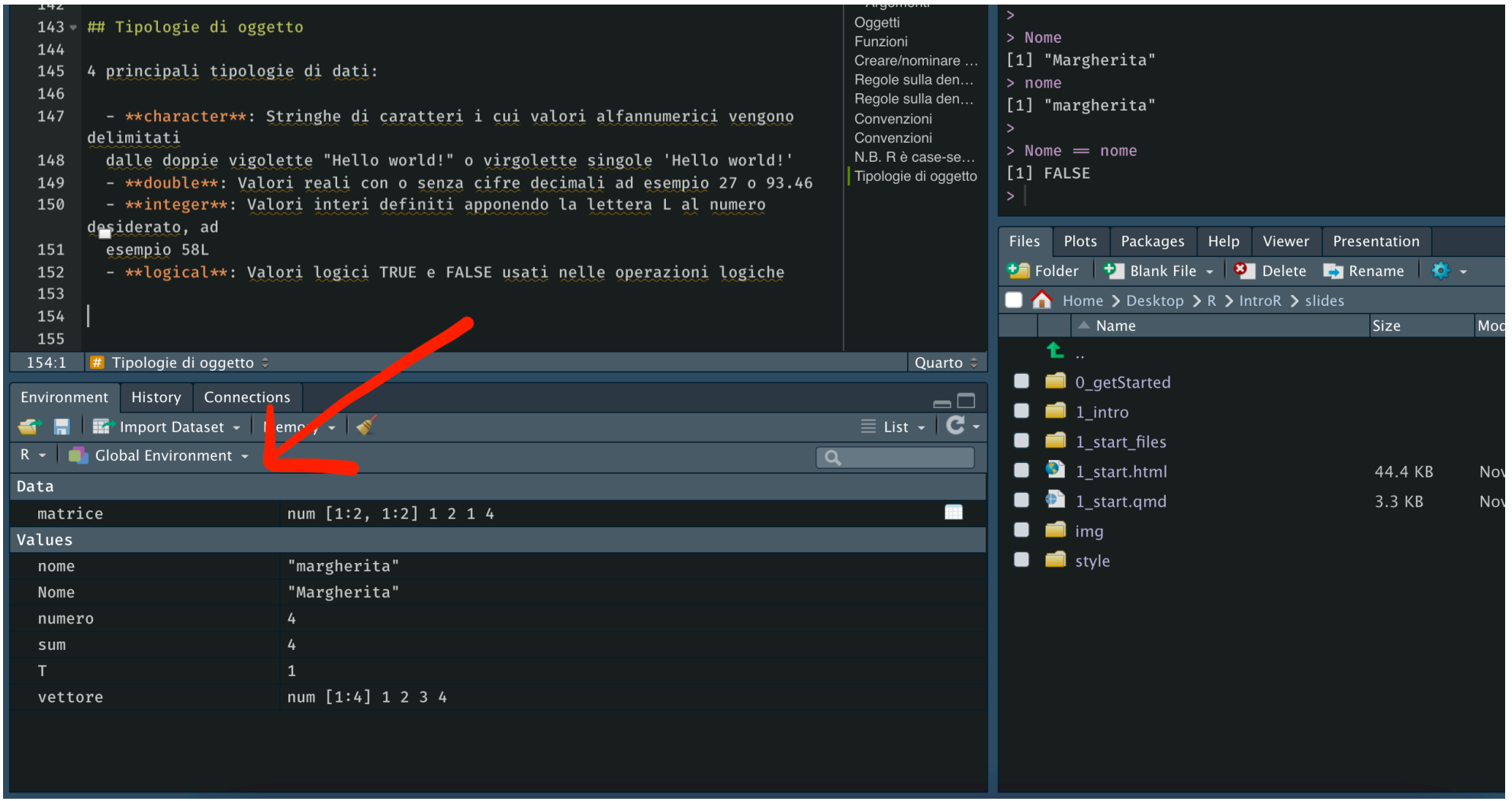
```
[1] "margherita"
```

```r
Nome == nome
```

```
[1] FALSE
```

Dove vengono salvati gli oggetti?

Di default gli oggetti sono creati nel **global environment** accessibile con `ls()` o visibile in R Studio con anche alcune informazioni aggiuntive:



Eliminare oggetti

Possiamo eliminare un oggetto presente nel nostro enviroment attraverso il comando `rm("nomeoggetto")`.

E’ possibile anche pulire completamente/svuotare il nostro enrivoment attraverso il comando `rm(list = ls())`.

Tipologie di dato

Tipologie di dato

- `character`: Stringhe di caratteri i cui valori alfanumerici vengono delimitati dalle doppie vigolette “Hello world!” o virgolette singole ‘Hello world!’
- `numeri`:
  - `double`: Valori reali con o senza cifre decimali ad esempio 27 o 93.46
  - `integer`: Valori interi definiti apponendo la lettera L al numero desiderato, ad esempio 58L

Operatori Matematici

Funzione	Cosa fa?	Esempio	Risultato
`+`	addizione	`5.4 + 6.1`	`11.5`
`-`	sottrazione	`9 - 4.3`	`4.7`
`*`	moltiplicazione	`7 * 1.4`	`9.8`
`/`	divisione	`9/3`	`3`
`%%`	resto	`9%%2`	`1`
`^`	potenza	`15 ^ 2`	`225`

Funzione	Cosa fa?	Esempio	Risultato
`abs`	valore assoluto	`abs(-8)`	`8`
`sqrt`	radice quadrata	`sqrt(225)`	`15`
`exp`	funzione esponenziale	`exp(0)`	`1`
`log`	logaritmo naturale	`log(1)`	`0`
`round`	arrotondamento, intero	`round(1.738)`	`2`
`round`	arrotondamento	`round(1.738, 2)`	`1.74`

Operazioni Matematiche

L’ordine delle operazioni in R segue le regole della matematica.

Esempi

```r
# Senza parentesi
1 + 2 * 3
```

```
[1] 7
```

```r
# Con le parentesi
(1 + 2) * 3
```

```
[1] 9
```

Operatori Relazionali

In R è possibile valutare se una data relazione è vera o falsa. R valuterà le proposizioni e ci restituirà il valore `TRUE` se la proposizione è vera oppure `FALSE` se la proposizione è falsa.

Funzione	Nome	Esempio	Risulato
`==`	uguale	`30 == 30`	`TRUE`
`!=`	diverso	`30 != 30`	`FALSE`
`>`/`>=`	maggiore/o uguale	`30 > 10`	`TRUE`
		`30 >= 10`	`TRUE`
`<`/`<=`	minore/o uguale	`30 < 10`	`FALSE`
		`10 <= 10`	`TRUE`
`%in%`	inclusione	`10%in%c(1,2,10)`	`TRUE`

Non vale solo per i numeri!

```
Nome = "Margherita"

nome = "margherita"

Nome == nome
```

```
[1] FALSE
```

PS. Ricordatevi che **=** è diverso da **==**

Operatori Logici

In R è possibile congiungere più relazioni per valutare una desiderata proposizione.

```
x = 30 #Assegnamo a x il valore 30.
```

Funzione	Nome	Esempio	Risulato
`&`	Congiunzione	`x>25 & x<60`	`TRUE`
`|`	Disgiunzione Inclusiva	`x>25 | x>60`	`TRUE`
`!`	Negazione	`!(x<18)`	`TRUE`

Operatori Logici

```
x = 30 #Assegnamo a x il valore 30.
x
```

```
[1] 30
```

```
x>25 & x>60
```

```
[1] FALSE
```

```
x>25 | x>60
```

```
[1] TRUE
```

```
!(x>18)
```

```
[1] FALSE
```

TRUE equivale a 1 e FALSE a 0

```
TRUE == 1
```

```
[1] TRUE
```

```
TRUE == 2
```

```
[1] FALSE
```

```
FALSE == 0
```

```
[1] TRUE
```

```
FALSE == 1
```

```
[1] FALSE
```

Creo un vettore esempio in cui ci sono dentro le parole casa (ripetuta 4 volte) e bottega (ripetuta 2 volte):

```
esempio = c(rep("casa",times = 4),rep("bottega", times = 2)) #?rep
esempio
```

```
[1] "casa"    "casa"    "casa"    "casa"    "bottega" "bottega"
```

```
# Quante parole dentro esempio sono uguali a casa?
esempio == "casa"
```

```
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE
```

```
sum(esempio == "casa") # perchè?
```

```
[1] 4
```

Ordine Valutazione Relazioni

Nel valutare le veridicità delle proposizioni R esegue le operazioni nel seguente ordine:

1. Operatori matematici (e.g., ^, *, /, +, -, etc.)
2. Operatori relazionali (e.g., <, >, <=, >=, ==, !=)
3. Operatori logici (e.g., !, &, |)

In caso di dubbi riguardanti l'ordine di esecuzione delle operazioni, la cosa migliore è utilizzare le parentesi tonde ( ) per disambiguare ogni possibile fraintendimento.

Facciamo un po’ di pratica!

Aprite e tenete aperto questo link: https://etherpad.wikimedia.org/p/arca-corsoR